

1. Dependencias funcionales y claves

Trabajo práctico — FNBC y desnormalización controlada

Grupo: Octavio Skumanic, Sebastián Vivanco, Victoria Guzman, Daniela Sosa, Guillermo Sanchez

La Parte 1 extiende Food Store con control de lotes en depósitos. La Parte 2 acelera el top 5 diario de categorías sobre el esquema real del proyecto. Los scripts, el informe y las evidencias se entregan juntos en **tpUnidad4/**; los trabajos anteriores permanecen conservados.

Parte 1 · Dependencias obtenidas del negocio

Sea $R(\text{LoteID}, \text{DepositoID}, \text{ResponsableControlID})$. Abreviamos L, D y R solo en las clausuras. El conjunto de dependencias es:

$\{\text{LoteID}, \text{DepositoID}\} \rightarrow \text{ResponsableControlID}$

$\text{ResponsableControlID} \rightarrow \text{DepositoID}$

La primera regla fija el responsable para cada lote y depósito. La segunda expresa su pertenencia a un único depósito, independientemente del lote.

Subconjunto X	Clausura X ⁺	¿Superclave?
{ } (vacío)	{ } (vacío)	No
L	L	No
D	D	No
R	R, D	No
L, D	L, D, R	Sí; mínima
L, R	L, R, D	Sí; mínima
D, R	D, R	No
L, D, R	L, D, R	Sí; no mínima

Conjunto completo de claves candidatas: $\{\text{LoteID}, \text{DepositoID}\}$ y $\{\text{LoteID}, \text{ResponsableControlID}\}$. LoteID no está en ningún lado derecho de las dependencias: toda clave debe contenerlo. L solo no alcanza; agregar D o R basta y produce las dos claves mínimas. Esto descarta claves adicionales.

Atributos primos: los tres, porque cada uno pertenece a alguna clave candidata. No hay atributos no primos.

Violación formal de FNBC

FNBC exige que el determinante de toda dependencia funcional no trivial sea superclave. $\text{ResponsableControlID} \rightarrow \text{DepositoID}$ es no trivial, pero $\{\text{ResponsableControlID}\}^+ = \{\text{ResponsableControlID}, \text{DepositoID}\}$, que no contiene LoteID . Por ello el esquema **no está en FNBC**. Que todos los atributos sean primos permite 3FN, pero no subsana esta violación de FNBC.

2. Anomalías y descomposición sin pérdida

Tres anomalías sobre la instancia provista

Inserción. Se necesita registrar que el responsable 803 pertenece al depósito 30 antes de asignarle un lote. La tabla original exige LoteID no nulo: no permite guardar ese dato maestro sin inventar un control. La prueba de INSERT con lote nulo obtiene SQLSTATE 23502. La tabla responsable_deposito sí permite registrarlo sin controles.

Borrado. La fila (503,31,802) es el único control del responsable 802. Al eliminarla desaparece también el único dato que indica su pertenencia al depósito 31, aunque el responsable siga existiendo en usuario. La prueba borra esa fila y comprueba que no queda ninguna asociación para 802; después hace ROLLBACK.

Actualización. El responsable 801 aparece en (501,30,801) y (502,30,801). Si se actualiza únicamente el depósito de la segunda fila a 31, el esquema original acepta que 801 pertenezca simultáneamente a dos depósitos. La prueba verifica los dos valores distintos y revierte el cambio.

Aplicación del algoritmo

Se elige $X = \{\text{ResponsableControlID}\}$ e $Y = \{\text{DepositoID}\}$. El algoritmo produce $R1 = X \text{ unión } Y$ y $R2 = R - (Y - X)$:

Relación resultante	Claves y referencias
responsable_deposito(R, D)	PK(R); FK(R) → usuario(id); FK(D) → deposito(id).
control_lote(L, R)	PK(L, R); FK(L) → lote(id); FK(R) → responsable_deposito(R).

R1 está en FNBC porque R determina todos sus atributos. En R2 no se proyectan dependencias funcionales no triviales que violen FNBC. Las tablas maestras y los tres registros originales se crean dentro del esquema unidad4, sin tocar las tablas públicas anteriores.

Unión sin pérdida. $R1 \cap R2 = \{\text{ResponsableControlID}\}$. Ese atributo común es clave de R1, pues $R \rightarrow RD$. Se satisface $(R1 \cap R2) \rightarrow R1$; por el criterio de descomposición binaria, la reunión es sin pérdida para las instancias que satisfacen las dependencias.

El SQL migra las proyecciones y reconstruye la relación mediante NATURAL JOIN, cuyo único atributo común es responsable_control_id. La comparación con EXCEPT ALL en ambos sentidos devuelve **cero filas**. La vista reconstruye exactamente las tres filas iniciales.

Preservación de dependencias. La reunión sin pérdida no implica preservar todas las dependencias localmente. $LD \rightarrow R$ requiere validar la reunión: se incorporan triggers por sentencia sobre ambas tablas, con serialización de escrituras, que rechazan dos responsables para un mismo lote y depósito. También se probó un conflicto entre dos conexiones.

3. Desnormalización y sincronización

Adaptación explícita al proyecto

El ejemplo de la consigna usa `id`, `subtotal` y `eliminado`; el esquema real usa `id_pedido`, `id_producto` e `id_categoria` y guarda cantidad y `precio_unitario`. Se calcula el subtotal histórico como su producto. Pedido y detalle no tienen baja lógica: se suman sus filas existentes. No se inventan esas columnas ni un filtro por estado. La baja de producto/categoría conserva su historia; por eso `activo=false` no elimina ventas del reporte.

El día comercial se define en `America/Argentina/Buenos_Aires` y se filtra entre medianoche inclusive y la siguiente exclusiva, sin comparar un `TIMESTAMP` directamente con `CURRENT_DATE`. La materializada guarda todos los días; el reporte elige el actual y resuelve empates por nombre. El nombre de categoría es único en el modelo.

Elección del patrón — justificación

Se elige una vista materializada por día y categoría porque la consulta normalizada midió una mediana de 114,230 ms y procesa 60.000 detalles del día mediante reuniones y agregación; la lectura precalculada midió 0,040 ms. Triggers por sentencia reconstruyen el resumen dentro de la transacción tras cambios en cualquiera de las cuatro fuentes, con escrituras serializadas y `READ COMMITTED`, de modo que los cambios y su resumen se confirman o revierten juntos. La decisión es reversible: al retirar los triggers y la materializada se conservan íntegramente las tablas fuente y el reporte normalizado.

Implementación y alcance operativo

`ventas_categoria_fuente` define la fuente de verdad. `ventas_categoria_diaria` materializa sus resultados. Un índice `UNIQUE(fecha,id_categoria)` identifica los grupos y otro índice `(fecha,total_vendido DESC,categoria)` favorece el top 5. El reporte optimizado consulta únicamente la materializada; no une las cuatro tablas originales.

Los triggers cubren `INSERT`, `UPDATE`, `DELETE` y `TRUNCATE` en `detalle_pedido`, `pedido`, `producto` y `categoria`. Incluyen cambios de fecha, reclasificación de productos, renombrado de categorías y desaparición de la última venta. Se usa `REFRESH` sin `CONCURRENTLY` y un advisory lock transaccional para serializar las escrituras. Las funciones son `VOLATILE`; las escrituras con otro aislamiento se rechazan explícitamente.

Costo y límites. Actualizar las seis líneas de un pedido pasó de 0,638 ms sin refresh a 209,583 ms con sincronización (medianas de cinco ensayos reversibles). La mejora de lectura no es gratuita: el refresh completo puede bloquear lectores hasta terminar la transacción. Es una elección para un panel con muchas lecturas y escrituras agrupadas, no una recomendación para cualquier carga OLTP. No se promete acceso sin bloqueos ni se oculta el costo de mantenimiento.

La auditoría compara todos los días y todas las columnas en ambos sentidos. Detectó inconsistencias inducidas al deshabilitar temporalmente el trigger; la prueba hizo `ROLLBACK`. Su ejecución final sobre la base migrada devuelve cero filas.

4. Medición antes y después

PostgreSQL 17.11, Windows x64, READ COMMITTED. Instancia sintética: 30 categorías, 600 productos, 20.000 pedidos y 120.000 detalles; 60.000 detalles corresponden al día. Se calentó cada consulta y se ejecutó EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) cinco veces. Se informa la mediana del tiempo de ejecución, sin planificación ni transferencia al cliente.

Antes — normalizado	Después — materializado
114,230 ms (mediana)	0,040 ms (mediana)
HashAggregate sobre una cadena de Hash Join; Seq Scan de 120.000 detalles.	Index Only Scan sobre u4_ventas_top y Limit; obtiene cinco filas.
El agregado concentra el costo acumulado de procesar las reuniones y agrupar.	Se evita reconstruir el agregado durante cada lectura.

Los tiempos de nodos son inclusivos: no se suman como si fueran independientes. En el plan anterior, HashAggregate tiene un costo estimado total de 4362,98 y su hijo Hash Join de 3762,60; estos valores son unidades del optimizador, no milisegundos. La captura es una ejecución adicional: su tiempo puede diferir de la mediana de las cinco repeticiones.

Plan posterior — salida real de PostgreSQL

```
Limit (cost=0.15..1.57 rows=5 width=20) (actual time=0.027..0.028 rows=5 loops=1)
  Buffers: shared hit=3
  -> Index Only Scan using u4_ventas_top on ventas_categoria_diaria (cost=0.15..8.67 rows=30
width=20) (actual time=0.026..0.027 rows=5 loops=1)
    Index Cond: (fecha = ((CURRENT_TIMESTAMP AT TIME ZONE
'America/Argentina/Buenos_Aires'::text))::date)
    Heap Fetches: 5
    Buffers: shared hit=3
Planning Time: 0.162 ms
Execution Time: 0.047 ms
```

La captura posterior muestra Heap Fetches: 5: Index Only Scan no significa que nunca consulte el heap. La captura anterior completa se incluye en la página siguiente. Los TXT y JSON originales están en la carpeta de evidencias.

Resultado funcional

66 comprobaciones correctas. Incluyen migración, FK, anomalías, regla LD → R, instalación reversible, altas/cambios/bajas, límites del día, TRUNCATE, rollback, auditoría negativa y concurrencia. Después de dos escrituras concurrentes confirmadas, ambos reportes siguen siendo iguales y la auditoría permanece vacía.

Las modificaciones de ambas sesiones quedan incluidas en los datos medidos. Los ensayos de escritura usados para cuantificar el refresh terminan en ROLLBACK. No se generalizan estos tiempos a otros equipos, volúmenes ni distribuciones.

5. Plan anterior completo

Salida real de EXPLAIN (ANALYZE, BUFFERS), exportada a imagen para el informe. Solo se ajustaron los saltos de línea para que el plan completo sea legible; no se alteraron nodos, tiempos ni cantidades.

```

Limit (cost=4363.48..4363.49 rows=5 width=45) (actual time=117.947..117.951 rows=5 loops=1)
  Buffers: shared hit=920
  -> Sort (cost=4363.48..4363.55 rows=30 width=45) (actual time=117.946..117.949 rows=5 loops=1)
    Sort Key: (sum(((dp.cantidad)::numeric * dp.precio_unitario))) DESC, c.nombre
    Sort Method: top-N heapsort  Memory: 25kB
    Buffers: shared hit=920
    -> HashAggregate (cost=4362.60..4362.98 rows=30 width=45) (actual time=117.920..117.931
rows=30 loops=1)
      Group Key: c.nombre
      Batches: 1  Memory Usage: 32kB
      Buffers: shared hit=920
      -> Hash Join (cost=1144.17..3762.60 rows=60000 width=22) (actual time=30.290..91.603
rows=60000 loops=1)
        Hash Cond: (pr.categoria_id = c.id_categoria)
        Buffers: shared hit=920
        -> Hash Join (cost=1142.50..3581.18 rows=60000 width=13) (actual
time=30.243..79.579 rows=60000 loops=1)
          Hash Cond: (dp.id_producto = pr.id_producto)
          Buffers: shared hit=919
          -> Hash Join (cost=1123.00..3403.07 rows=60000 width=13) (actual
time=30.013..65.885 rows=60000 loops=1)
            Hash Cond: (dp.id_pedido = ped.id_pedido)
            Buffers: shared hit=913
            -> Seq Scan on detalle_pedido dp (cost=0.00..1965.00 rows=120000
width=17) (actual time=0.013..7.902 rows=120000 loops=1)
              Buffers: shared hit=765
            -> Hash (cost=998.00..998.00 rows=10000 width=4) (actual
time=29.978..29.978 rows=10000 loops=1)
              Buckets: 16384  Batches: 1  Memory Usage: 480kB
              Buffers: shared hit=148
              -> Seq Scan on pedido ped (cost=0.00..998.00 rows=10000
width=4) (actual time=0.025..27.518 rows=10000 loops=1)
                Filter: (((fecha >= (((CURRENT_TIMESTAMP AT TIME ZONE
'America/Argentina/Buenos_Aires')::text)::date)::timestamp without time zone AT TIME ZONE
'America/Argentina/Buenos_Aires')::text)) AND (fecha < (((CURRENT_TIMESTAMP AT TIME ZONE
'America/Argentina/Buenos_Aires')::text)::date + 1))::timestamp without time zone AT TIME ZONE
'America/Argentina/Buenos_Aires')::text)))
                Rows Removed by Filter: 10000
                Buffers: shared hit=148
              -> Hash (cost=12.00..12.00 rows=600 width=8) (actual time=0.225..0.226
rows=600 loops=1)
                Buckets: 1024  Batches: 1  Memory Usage: 32kB
                Buffers: shared hit=6
                -> Seq Scan on producto pr (cost=0.00..12.00 rows=600 width=8)
(actual time=0.011..0.112 rows=600 loops=1)
                  Buffers: shared hit=6
                -> Hash (cost=1.30..1.30 rows=30 width=17) (actual time=0.040..0.040 rows=30
loops=1)
                  Buckets: 1024  Batches: 1  Memory Usage: 10kB
                  Buffers: shared hit=1
                  -> Seq Scan on categoria c (cost=0.00..1.30 rows=30 width=17) (actual
time=0.028..0.032 rows=30 loops=1)
                    Buffers: shared hit=1
Planning:
  Buffers: shared hit=24
Planning Time: 0.918 ms
Execution Time: 118.105 ms

```

6. Verificación y archivos de entrega

Resultados verificables

La ejecución de referencia está en `evidencias/20260930_190238_612510/`. resultado.json registra SQL, parámetros, salidas, errores esperados, entorno y las 66 comprobaciones. Su campo `exito` es `true`. La auditoría final contiene una lista vacía. Las filas del top 5 final son:

Categoría	Total vendido
Categoría 30	256108.00
Categoría 10	256061.00
Categoría 20	255837.00
Categoría 09	252162.00
Categoría 29	252136.00

Reproducción y entrega

Desde la raíz del repositorio, instalar `requirements.txt` y ejecutar `verificar.py` con `--pg-bin`, `--port` y `--user` de PostgreSQL. El verificador crea una base nueva con nombre único, genera datos sintéticos y guarda respaldos locales antes del DDL. Nunca reutiliza ni borra una base existente. El README explica la instalación alternativa sobre una copia ya poblada.

La consigna exige el enlace al repositorio con dos scripts y este informe. Se agregan el verificador y las evidencias para que el docente pueda repetir y contrastar los resultados. Los enlaces siguientes apuntan a los archivos de la carpeta `tpUnidad4` una vez publicada la actualización:

[Parte 1 — esquema, descomposición y migración](#)

[Parte 2 — materializada, sincronización, consultas y auditoría](#)

[Instrucciones de reproducción](#)

[Verificador y casos de prueba](#)

[Resultados y SQL ejecutado](#)

Uso de IA y decisiones

Se utilizó Codex para revisión, SQL, pruebas y documentación. Se aceptaron los cambios respaldados por ejecución. Se descartaron el trigger que solo contemplaba INSERT, las columnas inexistentes y la auditoría unilateral del borrador. No se declara uso de herramientas que no se utilizaron ni se presentan tiempos generados por IA como mediciones.

Referencias técnicas

[PostgreSQL 17: comportamiento de triggers](#)

[PostgreSQL 17: REFRESH MATERIALIZED VIEW](#)

[PostgreSQL 17: volatilidad y snapshots](#)